



[Outline](#) > 2. Improving the model > 2.3 Reaction time (Euler's Method) > Calculation (with Python)

Calculation (with Python)

🔖 Bookmark this page

Python program for Euler's Method

In the previous section Euler's Method was introduced, and now you can approximate the solution of a differential equation. Of course calculations by hand are inefficient. The method is very suitable to implement in a computer program: much faster and probably more accurate. So in this section, you will use Python to simulate the same solutions.

You can download the basic code for Euler's method here: `Basic_Euler.py`.

Run it, and see what it does!

Below, we will explain some details of the `Basic_Euler.py` program. Click one of the buttons if you want an answer to the question:

Question?

`Basic_Euler.py`: header and initialisations

```
# Program      : Euler's Method
```

Why write a header?

```
# Author       : MOOC team Mathematical Modelling Basics
```

```
# Created      : April, 2017
```

```
import numpy as np
```

Why import numpy?

```
import matplotlib.pyplot as plt
```

Why import matplotlib?

```
print("Solution for  $dP/dt = 0.7 \cdot P$ ")
```

Not using Python 3.6?

Necessary statement?

Typesetting math: 0%

```

Dt = 0.1          # timestep Delta t
P_init = 10       # initial population
t_init = 0        # initial time
t_end = 5         # stopping time
n_steps = int(round((t_end-t_init)/Dt)) # total number of timesteps

```

Why use comments?

```
P_arr = np.zeros(n_steps + 1) # create an array of zeros for P
```

Why create an array with zeros?

```
t_arr = np.zeros(n_steps + 1) # create an array of zeros for t
```

Why not fill this array?

```
t_arr[0] = t_init # add the initial t to the array
```

= ?

```
P_arr[0] = P_init # add the initial P to the array
```

Basic_Euler.py: Iterations Euler's Method

```

for i in range (1, n_steps + 1):
    P = P_arr[i-1]
    t = t_arr[i-1]
    dPdt = 0.7*P # calculate the derivative
    P_arr[i] = P + Dt*dPdt # calculate P on the next time step
    t_arr[i] = t + Dt # adding the new t-value to the list

```

Is this necessary?

Basic_Euler.py: Plotting the results

```

fig = plt.figure() # create figure
plt.plot(t_arr, P_arr, linewidth = 4) # plot population vs. time

```

```
plt.title('dP/dt = 0.7P, P(0)=10', fontsize = 25)
```

Large font?

```
plt.xlabel('t (in days)', fontsize = 20)
```

```
plt.ylabel('P(t)', fontsize = 20)
```

```
plt.xticks(fontsize = 15)
```

```
plt.yticks(fontsize = 15)
```

```
plt.grid(True) # show grid
```

```
plt.axis([0, 5, 0, 200]) # define the axes
```

```
plt.show() # show the plot
```

```
# save the figure as .jpg
```

```
fig.savefig('Rainbowfish.jpg', dpi=fig.dpi, bbox_inches = "tight")
```

Typesetting math: 0%

As you have seen in the previous section, a large time step of (7) days leads to results that are not valid for the rainbowfish population. In the next video you will see what happens when the stepsize is reduced.

Video: Choosing a stepsize for Euler's Method



0:00 / 7:45



2.0x



Video ▼



Subtitles ▼

Other ▼

[CC BY-NC-SA](#) Some Rights Reserved

Subtitles (captions) in other languages than provided can be viewed at YouTube. Select your language in the CC-button of YouTube.

Typesetting math: 0%

In the video you have seen that a step size of 1 day is not too large to obtain reasonable results. Now it is time for you to do some programming yourself.

Our problem:

Find t_{eq} such that $P(t_{\text{eq}}) = 720$, when

$$\frac{dP}{dt} = 0.7P \left(1 - \frac{P}{750}\right) - 20, \quad P(0) = 30.$$

In the exercises you will be asked to give the values your computer program returns. You can see those values (for example for index=21) by typing in the IPython console: `t_arr[21]` and `P_arr[21]`. Alternatively, you can put print statements in your program and run it (again).

Exercise A

1/1 point (graded)

Change the code so that it computes approximate solutions using Euler's Method for the differential equation for the rainbowfish. Use a stepsize of 1 day and simulate until 60 days.

Give the value of the rainbowfish population after 1 day.

Give your answer in two decimals.



$\backslash(\backslash)$

You have used 1 of 10 attempts

Exercise B

1/1 point (graded)

For this exercise, use the same code as in the previous exercise. So $\Delta t = 1$ day.

Give the value of the rainbowfish population after 13 days: $P(13) \approx$

Give your answer in two decimals.



$\backslash(\backslash)$

You have used 1 of 10 attempts

Typesetting math: 0%

Exercise C

1/1 point (graded)

Now use a smaller stepsize: $\Delta t = 0.5$, and approximate the population after 13 days.

$P(13) \approx$

Give your answer in two decimals.



$\backslash \backslash$

You have used 3 of 10 attempts

Exercise D

1/1 point (graded)

Now use an even smaller stepsize: $\Delta t = 0.25$, and approximate the population now after 13 days.

$P(13) \approx$

Give your answer in two decimals.

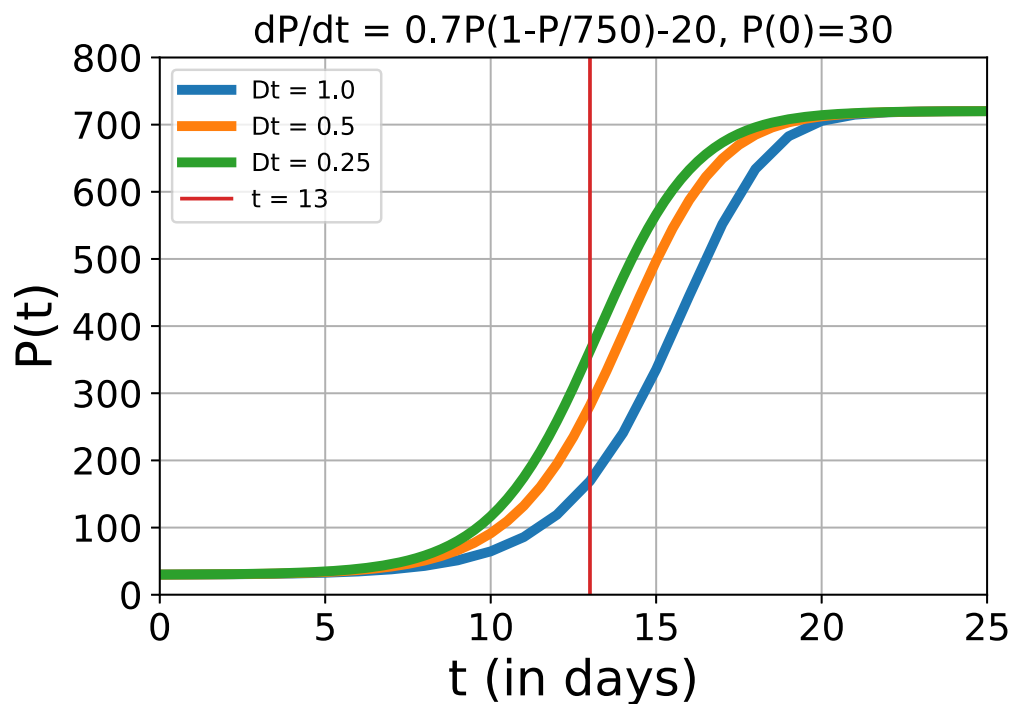


$\backslash \backslash$

You have used 1 of 10 attempts

For this exercise, recreate the following figure:

Typesetting math: 0%



You will have to store the results for the three stepsizes with different names. A low-tech way to do that is:

Repeat three times:

1. Run the program for stepsize x ,
2. Type in the IPython console: `t_ssx = np.copy(t_arr)` and `P_ssx = np.copy(P_arr)`.

Why not use `t_ssx = t_arr` ?

Then plot the three curves by just using `plt.plot` three times in the same figure.

(A more professional way would be to write yourself a function with the stepsize as input, and both arrays as output, and then call the function three times.)

To plot the vertical red line, you could use: `plt.plot([13,13],[0,750], 'r', linewidth=2)`. Do you see how that works?

Exercise E

1/1 point (graded)

Did you recreate the (important parts of the) figure?

☐ No

Typesetting math: 0%

[Submit](#)

You have used 1 of 10 attempts

Now that you have practised with Python, it is time to look into the accuracy of the results. See the next page.

© All Rights Reserved



© 2012-2017 edX Inc. All rights reserved except where noted. EdX, Open edX and the edX and Open EdX logos are registered trademarks or trademarks of edX Inc. | 粵ICP备17044299号-2

POWERED BY
OPENedX[®]

Typesetting math: 0%